

Contents

1	Introduction	2
2	Agent Internal Architecture	3
2.1	BDI Architecture Overview	3
2.2	Traffic Monitor Agent (TMA) — BDI Model	3
2.3	Anomaly Classifier Agent (ACA) — BDI Model	4
2.4	Response Coordinator Agent (RCA) — BDI Model	5
2.5	Threat Intelligence Agent (TIA) — BDI Model	6
2.6	Resource Allocator Agent (RAA) — BDI Model	7
2.7	Attacker Agents — BDI Model	8
3	Interaction & Communication Design	10
3.1	Communication Protocol — FIPA-ACL	10
3.1.1	Message Schema	10
3.1.2	Topic Registry	10
3.1.3	Asynchronous Synchronization Mechanism	10
3.2	Message Payload Definitions	11
3.2.1	Alert Message (TMA → ACA)	11
3.2.2	Threat Report (ACA → RCA, TIA)	11
3.2.3	Resource Bid (Agent → RAA)	12
3.2.4	Vote Request / Response (RCA ↔ Coalition)	12
3.2.5	Failure Message (Any Agent → Sender of failed request)	12
3.2.6	Not-Understood Message (Any Agent → Sender of malformed message)	12
3.3	Sequence Diagrams	13
3.3.1	Single-Segment Detection & Response	13
3.3.2	Coalition Formation (Multi-Segment Attack)	13
3.3.3	Auction-Based Resource Allocation	14
3.3.4	Voting Protocol for Quarantine	15
4	Coordination Mechanism Logic	17
4.1	Proportional Response Selection	17
4.2	Auction-Based Resource Allocation	17
4.3	Coalition Formation Protocol	19
4.4	Voting-Based Quarantine Escalation Protocol	20
4.4.1	Voting Protocol	20
4.4.2	Voting Decision Rule Per Agent	20
4.5	Agent Failure & Resilience Protocol	21
5	System Implementation Design	22
5.1	Technical Stack	22
5.2	UML Class Diagram	22
5.2.1	BDI Hierarchy	23
	Agent Class Diagram	24
5.3	Deployment Architecture	25
6	Simulator & Visualization Architecture	26
6.1	Simulation Engine Design	26
6.1.1	Core Components	26
6.1.2	Simulation API Endpoints	27
6.2	Visualization	27

6.2.1	Dashboard Page	27
6.2.2	Agent Inspector Page	29
6.2.3	Validator Page	31
7	Data & Analytics Design	34
7.1	Event Log Schema	34
7.2	Metric Collection Design	35
7.3	Validation Scenario Reporting	35
7.4	Scenario Success Criteria Summary	36
8	Summary	37
9	Appendix: Glossary & Acronyms	38

1 Introduction

This System Design Document serves as the technical blueprint for the Collaborative Cyber-Defense and Automated Incident Response Multi-Agent System (MAS). It details the internal agent architectures, communication protocols, and coordination algorithms necessary for implementation. This document is structured to provide the technical depth and simulator feasibility required to meet all engineering standards and project criteria.

2 Agent Internal Architecture

All agents in the system are designed using the BDI (Belief-Desire-Intention) cognitive architecture. This model provides a principled separation between what an agent knows (Beliefs), what it wants to achieve (Desires), and the concrete plans it commits to executing (Intentions). This architecture ensures that agents behave rationally, predictably, and in accordance with their utility functions.

2.1 BDI Architecture Overview

Each agent consists of the following core components:

Component	Description
Belief Base	The agent's current knowledge about the world, updated via perception.
Desires (Goal Set)	The agent's motivation. States what it wants to achieve. Goals persist until satisfied or dropped.
Plan Library	A repository of pre-defined conditional plans. Each plan is a tuple (trigger-condition, pre-conditions, action-sequence). Agents select the plan best matching their active goals.
Intention Stack	The currently executing plans. An agent selects from applicable plans and commits to execution.

Table 1 – BDI Architecture Overview.

Each agent runs a continuous perceive → reason → act loop. The loop is event-driven (triggered by a message arrival on a subscribed topic or by a periodic timer), which bridges the gap between the agent's discrete reasoning cycle and the continuous nature of the network environment.

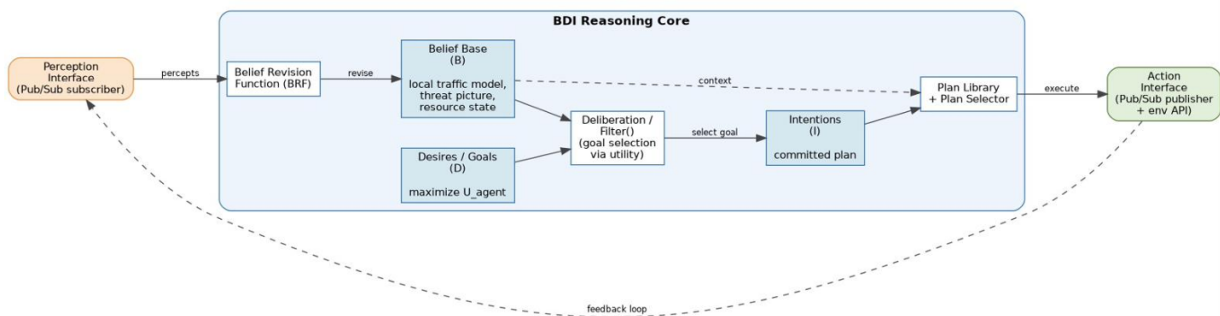


Figure 1 – BDI Architecture Overview.

2.2 Traffic Monitor Agent (TMA) — BDI Model

Beliefs

- **segment_id**: string — the assigned network segment (e.g. public-facing).
- **current_traffic**: src_ip, dst_ip, port, protocol, pkt_size, freq, timestamp.
- **baseline_mean**: float — rolling average of traffic rate over the last 60 seconds.
- **baseline_std**: float — standard deviation of traffic rate.
- **last_alert_time**: timestamp — prevents duplicate alert flooding.
- **resource_available**: boolean — whether extra resources have been granted.

Desires

- Maximize Detection Rate for its assigned segment.
- Minimize False Positive Rate below 10%.
- Publish alerts within 100 ms of anomaly occurrence.

Plans

Detect Anomaly

- Trigger: $\text{current_rate} > \text{baseline_mean} + 2 \times \text{baseline_std}$.
- Execution Steps:
 - Compute deviation score.
 - Construct alert message.
 - Publish to alert topics.
 - Update `last_alert_time`.

Update Baseline

- Trigger: $\text{now} - \text{last_baseline_update} \geq 60 \text{ s}$.
- Execution Steps:
 - Compute new mean from traffic window.
 - Compute new standard deviation.
 - Update belief base.
 - Log update.

Request Resources

- Trigger: $\text{current_traffic.volume} > \text{HIGH_THRESHOLD}$.
- Execution Steps:
 - Compute required DPI capacity.
 - Submit bid to RAA via `resource-bids` topic.
 - Await allocation notification.

2.3 Anomaly Classifier Agent (ACA) — BDI Model

Beliefs

- `alert_queue`: `Alert[]` — received but unprocessed alerts from Traffic Monitor Agent (TMA).
- `classification_model`: local ML model with known attack signatures.
- `threat_intel`: `ThreatReport[]` — shared intelligence from Threat Intelligence Agent (TIA) and peer ACAs.
- `incident_history`: `IncidentRecord[]` — resolved incidents used for learning.
- `false_positive_rate`: `float` — must stay below 0.10.

Desires

- Classify every alert within 200 ms of receipt.
- Maintain classification accuracy above 90% and FPR below 8%.
- Continuously improve model accuracy via online learning after resolved incidents.

Plans

Classify Alert

- Trigger: `alert_queue  $\neq \emptyset$   $\wedge$  (now - alert.received_time < 200 ms).`
- Execution Steps:
 - Extract high-dimensional features from the incoming alert.
 - Execute `model.predict(features)` via the local inference engine.
 - Assign a severity score within the range [0.0, 1.0].
 - Publish the resulting threat report to the `threat-reports` topic.
 - Dequeue the alert from the processing buffer.

Online Learning

- Trigger: Receipt of `incident_resolved` event.
- Execution Steps:
 - Extract the feature-label pair (x, y) from the resolved incident data.
 - Update internal cluster centroids or decision boundaries using the incremental learner.
 - Log the `model_improvement_rate` delta for performance tracking.
 - Broadcast updated intelligence to the MAS if the learning delta exceeds the significance threshold.

Share Intelligence

- Trigger: `novelty_score > novelty_threshold`.
- Execution Steps:
 - Generate a concise summary of the newly identified threat pattern.
 - Publish the pattern metadata to the `threat-intel` topic.
 - Update the peer belief synchronization vector to ensure global MAS alignment.

2.4 Response Coordinator Agent (RCA) — BDI Model

Beliefs

- `confirmed_threats`: `ThreatReport[]` — active threats with severity ≥ 0.7 .
- `available_actions`: `ResponseAction[]` — type: `THROTTLE` | `BLOCK` | `REDEPLOY` | `QUARANTINE`, `estimated_effectiveness`, `disruption_score`.
- `resource_allocation`: `ResourceGrant` — current resources granted by RAA.
- `coalition_members`: `AgentID[]` — current coalition (if active).
- `vote_results`: `VoteRecord` — latest voting round outcome.
- `incident_log`: `IncidentRecord[]` — persistent log of all decisions.

Desires

- Initiate defensive response within 500 ms of receiving a Confirmed Threat with severity ≥ 0.7 .
- Maximize system availability and minimize `MTTR_Response`.
- Ensure all quarantine actions pass majority coalition vote before execution.
- Select the least disruptive effective response action (proportionality).

Plans

Respond To Threat

- Trigger: `confirmed_threat ∧ (severity ≥ 0.7) ∧ ¬response_initiated`.
- Execution Steps:
 - Select a proportional action based on the policy.
 - If `action == QUARANTINE`: call `initiate_voting()`; else call `execute_action()` for immediate mitigation.
 - Log the selected decision and rationale to the local audit trail.
 - Monitor the outcome to verify mitigation effectiveness.

Initiate Voting

- Trigger: `action_type == QUARANTINE`.
- Execution Steps:
 - Broadcast a `VoteRequest` message to all agents in the current coalition.
 - Initialize a timeout timer for 300 ms.
 - Collect and aggregate incoming votes from peers.
 - If a majority consensus is reached, call `execute_quarantine()`; otherwise fall back to the `BLOCK` action as a safety default.
 - Log the final voting tally and the resulting outcome.

Resolve Incident

- Trigger: Confirmation of `threat_neutralized` state.
- Execution Steps:
 - Broadcast a `ResolutionNotice` to the coalition to sync global state.
 - Release all held resource claims back to the Resource Allocation Agent (RAA).
 - Finalize the incident log with a full event timeline.
 - Dissolve the local coalition if the Threat Intelligence Agent (TIA) confirms no residual risk.

2.5 Threat Intelligence Agent (TIA) — BDI Model

Beliefs

- `global_threat_map`: `Map<SegmentID, ThreatReport[]>` — aggregated threat reports from all ACAs across all segments.
- `correlation_matrix`: `float[][]` — statistical correlation between threat patterns across segment pairs.
- `external_threat_feed`: `SignatureDB` — simulated external signatures representing known attack patterns.
- `incident_history`: `HistoricalIncident[]` — long-term incident records for pattern learning.
- `active_coalitions`: `Coalition[]` — currently active coalitions and their members.

Desires

- Maintain a global threat model updated at least every 500 ms.

- Detect multi-segment correlations within 1 second of co-occurring threat reports.
- Publish priority-ranked threat list every 1 second.
- Trigger coalition formation within 1,000 ms of detecting a confirmed multi-segment incident.

Plans

Update Threat Model

- Trigger: $\Delta t \geq 500$ ms (Periodic Timer).
- Execution Steps:
 - Consume all pending `THREAT_REPORT` messages from the input buffer.
 - Update the `global_threat_map` with the latest telemetry and telemetry signatures.
 - Generate and publish an enriched `INTEL_SUMMARY` to the global agent bus.

Detect Correlation

- Trigger: $\text{count}(\text{flagged_segments}) \geq 2$ within $t < 2.0$ s.
- Execution Steps:
 - Execute the correlation scoring algorithm on the current correlation graph.
 - If score > threshold, escalate the state to `MULTI_SEGMENT_INCIDENT`.
 - Broadcast a `COALITION_INVITE` to relevant peer agents to initiate collective defense.

Rank Threats

- Trigger: $\Delta t \geq 1000$ ms (Periodic Timer).
- Execution Steps:
 - Sort the entries in the `global_threat_map` by their calculated severity score in descending order.
 - Compile the top-ranked threats into a `PRIORITY_LIST`.
 - Publish the `PRIORITY_LIST` to the system bus for consumption by the Resource Allocation Agent (RAA) and all Response Coordination Agents (RCAs).

2.6 Resource Allocator Agent (RAA) — BDI Model

Beliefs

- `resource_pool`: `ResourcePool` — total available DPI slots, quarantine slots, CPU budget.
- `active_bids_queue`: `Bid[]` — pending bids received from agents.
- `host_utilization`: `float` — current CPU + memory usage of MAS vs host capacity.
- `active_allocations`: `Allocation[]` — currently granted resource assignments.
- `resolved_incidents`: `IncidentID[]` — incidents confirmed resolved, triggering resource reclamation.

Desires

- Allocate resources exclusively to agents with active threats, prioritizing severity ≥ 0.7 .
- Complete sealed-bid auction within 300 ms of receiving competing bids.
- Keep total MAS resource overhead below 40% of host capacity.
- Reclaim and redistribute resources within 500 ms of receiving a `RESOLUTION_NOTICE`.

- Notify all bidders within 100 ms of auction close.

Plans

Run Auction

- Trigger: $\text{count}(\text{RESOURCE_BIDs}) \geq 2$.
- Execution Steps:
 - Sort incoming bids based on the threat severity score S in descending order.
 - Allocate resources to top-tier bidders sequentially until the `resource_pool` is exhausted.
 - Broadcast the `AUCTION_RESULT` to all participating agents within a strict 100 ms latency window.
 - Update the `active_allocations` table to reflect current system state.

Reclaim Resources

- Trigger: Receipt of a `RESOLUTION_NOTICE` message.
- Execution Steps:
 - Remove the notifying agent's entry from the `active_allocations` table.
 - Return the freed capacity to the global `resource_pool`.
 - Check the pending bid queue: if `queue_length > 0`, immediately trigger the `RUN_AUCTION` plan.

Monitor Overhead

- Trigger: $\Delta t \geq 1000$ ms (Periodic Timer).
- Execution Steps:
 - Sample current `host_utilization` metrics (CPU/RAM).
 - **Warning State:** if utilization $> 35\%$, generate a system warning log.
 - **Critical State:** if utilization $> 40\%$, initiate a forced reclamation of the lowest-priority allocation to maintain simulator stability.

2.7 Attacker Agents — BDI Model

Beliefs

- `target_map`: `SegmentProfile[]` — known network segments with observed traffic baselines, open ports, and inferred service types.
- `defense_state`: `DefenseObservation` — inferred current state of the defense coalition (response latency, active blocks, quarantine zones).
- `attack_playbook`: `AttackStrategy[]` — available attack strategies with estimated disruption impact and detection risk per strategy.
- `evasion_history`: `EvasionRecord[]` — log of past attack attempts annotated with whether a defensive response was triggered and at what latency.
- `evasion_rate`: float — fraction of attack duration spent unmitigated, must stay > 0.85 to consider the campaign successful.

Desires

- Maximize `disruption_impact` on protected services while remaining undetected for as long as possible.
- Maximize the attacker utility $U_{\text{ATK}} = \text{disruption} \times \text{duration_unmitigated}$.
- Maintain persistence even if individual nodes are quarantined.

Plans

Reconnaissance Scan

- Trigger: `target_landscape` is incomplete or $\Delta t \geq 5.0$ s.
- Execution Steps:
 - Perform low-intensity TCP/UDP port scanning on a randomized subset of the IP range.
 - Fingerprint services to identify versions (e.g. OpenSSH 7.2p2).
 - Cross-reference services against the `exploit_library` to identify potential entry points.
 - Update the `target_landscape` belief base with identified vulnerabilities.

Execute Attack

- Trigger: `target_map` $\neq \emptyset \wedge \text{current_phase} = \text{ATTACK}$.
- Execution Steps:
 - Select the highest-yield target segment from `target_map` based on estimated `disruption_impact` score.
 - Retrieve the corresponding attack strategy from `attack_playbook` (e.g. DDoS flood, port scan, lateral move).
 - Execute the attack action against the target segment with the configured intensity and timing parameters.
 - Record the action in `evasion_history` with timestamp, target, traffic volume, and `attack_type`.

3 Interaction & Communication Design

The system adopts **FIPA-ACL** (Foundation for Intelligent Physical Agents — Agent Communication Language) as its inter-agent communication standard. FIPA-ACL defines a structured message format with a performative (speech act) that expresses the communicative intent, enabling agents to distinguish between informing, requesting, proposing, and voting without parsing unstructured text.

3.1 Communication Protocol — FIPA-ACL

3.1.1 Message Schema

All messages exchanged on the publish-subscribe bus conform to the following JSON-based envelope:

```
{
  "msg_id":      "<UUIDv4>",
  "performative": "INFORM | REQUEST | PROPOSE | ACCEPT | REJECT |
                  CALL-FOR-PROPOSAL | BID | FAILURE | NOT-UNDERSTOOD",
  "sender":      "<agent_type>:<agent_id>",
  "receiver":    "<agent_type>:<agent_id> | BROADCAST",
  "topic":       "alerts | threat-reports | threat-intel |
                  resource-bids | votes | coalition | resolution",
  "conversation-id": "<uuid>",
  "reply-by":     "<ISO-8601 timestamp>",
  "content":      { <payload, defined per performative> },
  "timestamp":    "<ISO-8601 timestamp>"
}
```

3.1.2 Topic Registry

Topic	Publisher(s)	Subscriber(s)	Message Types
alerts	TMA	ACA	INFORM
threat-report	ACA	RCA, TIA	INFORM
threat-intel	TIA, ACA	ACA, RCA, RAA	INFORM
resource-bids	TMA, ACA, RCA	RAA	CALL-FOR-PROPOSAL / BID
resource-gran	RAA	All agents	INFORM
coalition	TIA	ACA, RCA, RAA	REQUEST / INFORM
votes	RCA	Coalition members	PROPOSE / ACCEPT / REJECT
resolution	RCA	All coalition members, RAA	INFORM

Table 2 – Communication Protocol — Topic Registry.

3.1.3 Asynchronous Synchronization Mechanism

Due to the environment being continuous and non-stationary, two Traffic Monitor Agents in different segments may publish alerts at the same wall-clock instant. Without ordering guarantees, the Anomaly Classifier could classify an effect before its cause, or a vote could close before a late

ballot arrives. These race conditions are eliminated by three coordinated mechanisms:

- **Lamport logical clocks:** Every agent maintains a counter incremented on each send and on receive (taking the maximum of local and incoming). The bus stamps each message and establishes a total order via the tuple (`lamport_ts`, `sender_id`), breaking ties deterministically. Causally related events can never be delivered out of order.
- **Per-topic FIFO queues with single-consumer locks:** Each topic is an `asyncio.Queue` guarded by an `asyncio.Lock`. Delivery into a topic is serialized, so subscribers observe a consistent, monotonic sequence per topic even though publishers run concurrently as coroutines.
- **Idempotent deduplication:** The bus and every subscriber track the highest (`sender`, `seq`) seen. A retried or duplicated message (for example, after agent recovery) is dropped silently, so re-publication during failover never double-triggers a response.
- **FAILURE / NOT-UNDERSTOOD error propagation:** Any agent that receives a malformed message or cannot execute a requested action publishes a FAILURE or NOT-UNDERSTOOD reply to the originating agent. This ensures that failed operations surface immediately rather than silently timing out, and allows the sending agent to trigger its fall-back plan within the same BDI cycle.

Time-bounded interactions (200ms coalition reply, 300ms auction or vote) are enforced with `asyncio.wait_for` deadlines measured against the bus clock. A non-responding agent is treated as a refuse so the protocol always terminates within its timing budget rather than blocking indefinitely.

3.2 Message Payload Definitions

3.2.1 Alert Message (TMA → ACA)

```
{
  "alert_id":      "uuid",
  "segment":      "public-facing | server | internal | sec-mon",
  "anomaly_type":  "VOLUME_SPIKE | PORT_SCAN | LATERAL | NOVEL",
  "deviation_score": 4.2,                // standard deviations from baseline
  "src_ips":      ["10.0.1.x", ...],
  "dst_port":     443,
  "traffic_volume": 9800,                // packets/sec
  "baseline_mean": 400,
  "baseline_std":  50,
  "detected_at":   "2026-05-05T10:00:00.123Z"
}
```

3.2.2 Threat Report (ACA → RCA, TIA)

```
{
  "threat_id":      "uuid",
  "alert_id":      "uuid",
  "classification": "NORMAL | SUSPICIOUS | CONFIRMED_THREAT",
  "attack_type":    "DDOS | PORT_SCAN | LATERAL | RANSOMWARE | ZERO_DAY",
  "severity":       0.91,                // 0.0 to 1.0
  "segment":       "public-facing",
  "confidence":     0.87,
  "classified_at":  "2026-05-05T10:00:00.290Z"
}
```

3.2.3 Resource Bid (Agent → RAA)

```
{
  "bid_id":          "uuid",
  "bidder_id":       "RCA:1",
  "resource_type":   "DPI_SLOT | QUARANTINE_SLOT | CPU_BUDGET",
  "quantity":        1,
  "bid_value":        0.91,          // = threat severity score (FR auction
    rule)
  "justification_threat_id": "uuid",
  "submitted_at":     "2026-05-05T10:00:00.350Z"
}
```

3.2.4 Vote Request / Response (RCA ↔ Coalition)

```
// VoteRequest
{
  "vote_id":         "uuid",
  "proposal":         "QUARANTINE",
  "target_segment":   "server-zone",
  "threat_id":        "uuid",
  "severity":         0.85,
  "deadline":         "2026-05-05T10:00:00.650Z"
}

// VoteResponse
{
  "vote_id":         "uuid",
  "voter_id":         "ACA:2",
  "decision":         "ACCEPT | REJECT",
  "rationale":        "local_threat_score=0.88, supports_quarantine=true"
}
```

3.2.5 Failure Message (Any Agent → Sender of failed request)

```
{
  "failure_id":       "uuid",
  "in_reply_to":      "uuid",
  "sender":           "RCA:1",
  "receiver":         "RAA:1",
  "reason":           "TIMEOUT | RESOURCE_UNAVAILABLE | PLAN_PRECONDITION_FAILED",
  "failed_action":     "QUARANTINE",
  "threat_id":        "uuid",
  "description":       "Vote deadline exceeded; quarantine not executed. Falling back
    to BLOCK.",
  "fallback_action":   "BLOCK",
  "timestamp":         "2026-05-05T10:00:01.000Z"
}
```

3.2.6 Not-Understood Message (Any Agent → Sender of malformed message)

```
{
  "not_understood_id": "uuid",
  "in_reply_to":       "uuid",
}
```

```

"sender":      "ACA:2",
"receiver":    "TMA:1",
"reason":      "UNKNOWN_PERFORMATIVE | SCHEMA_VIOLATION | MISSING_FIELD",
"offending_field": "anomaly_type",
"description": "Received alert with anomaly_type='UNKNOWN', not in the
               accepted enum. Message dropped.",
"timestamp":   "2026-05-05T10:00:00.005Z"
}

```

3.3 Sequence Diagrams

3.3.1 Single-Segment Detection & Response

The following sequence describes the end-to-end pipeline from anomaly detection through response execution.

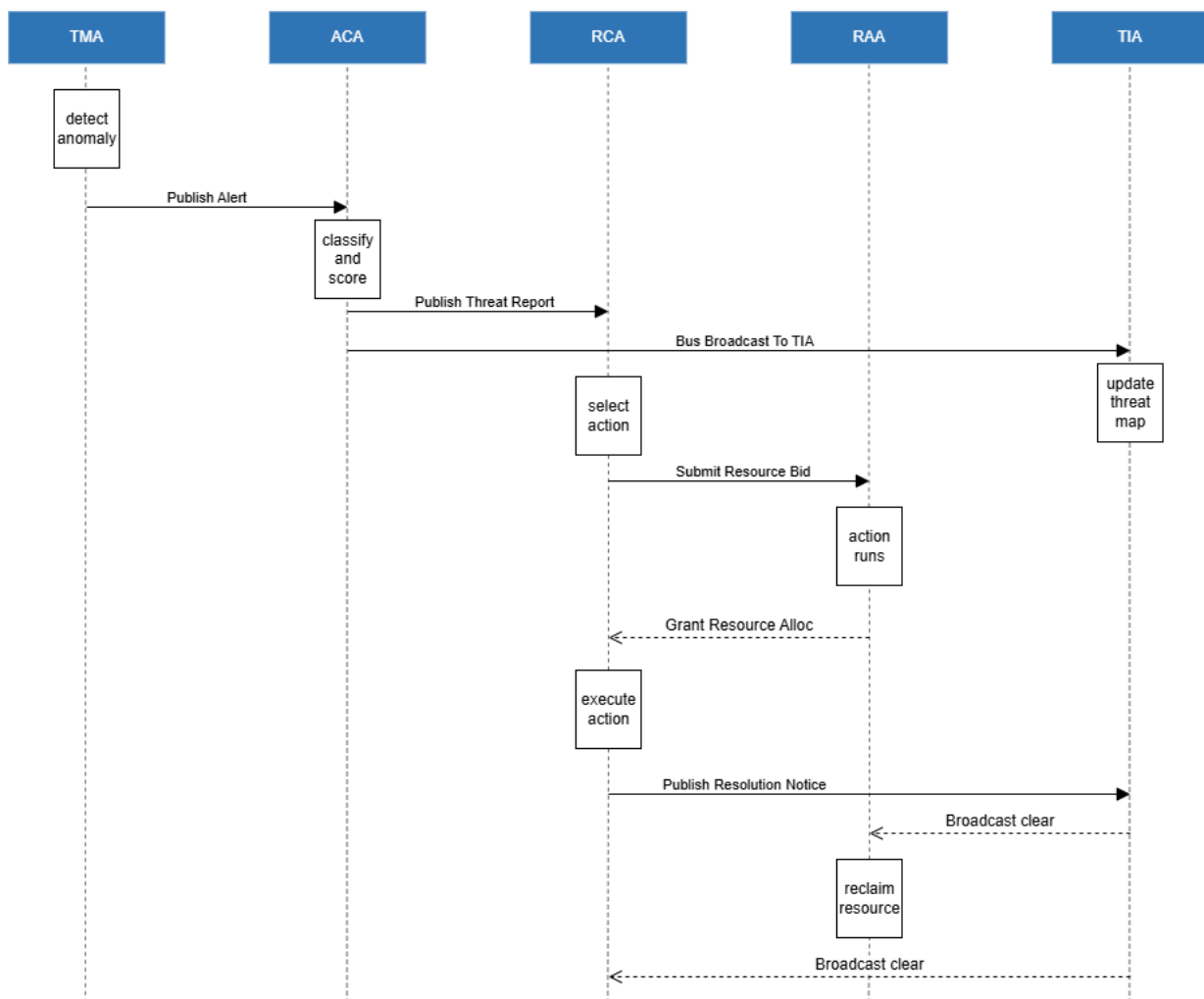


Figure 2 – Single-Segment Detection & Response Sequence Diagram.

3.3.2 Coalition Formation (Multi-Segment Attack)

This sequence covers the TIA-mediated coalition formation.

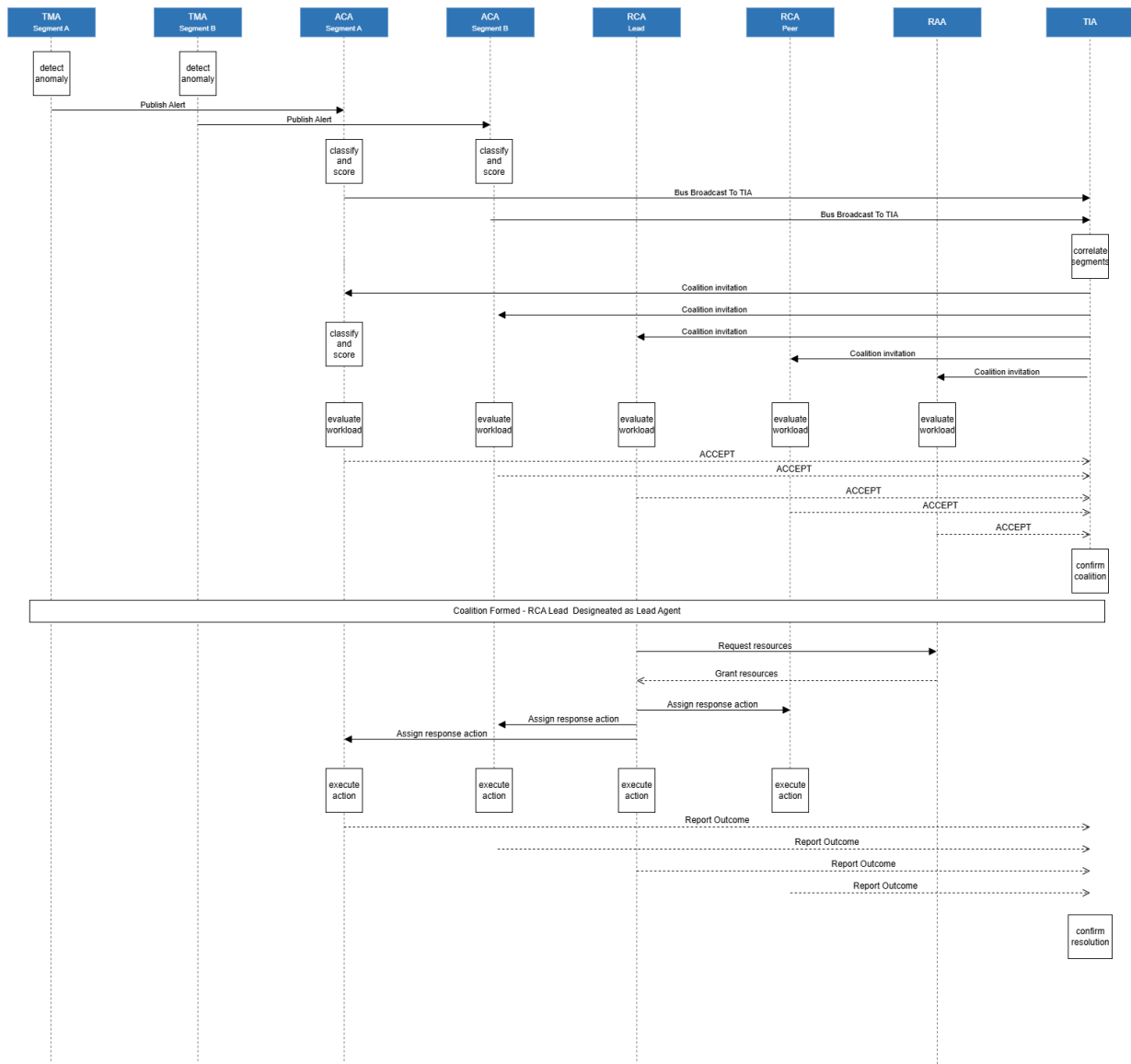


Figure 3 – Coalition Formation (Multi-Segment Attack) Sequence Diagram. Full diagram: [link](#).

3.3.3 Auction-Based Resource Allocation

Resource allocation uses a sealed-bid, first-price auction managed by the Resource Allocator. The bid value equals the threat severity score, so the highest-severity threat wins contested resources.

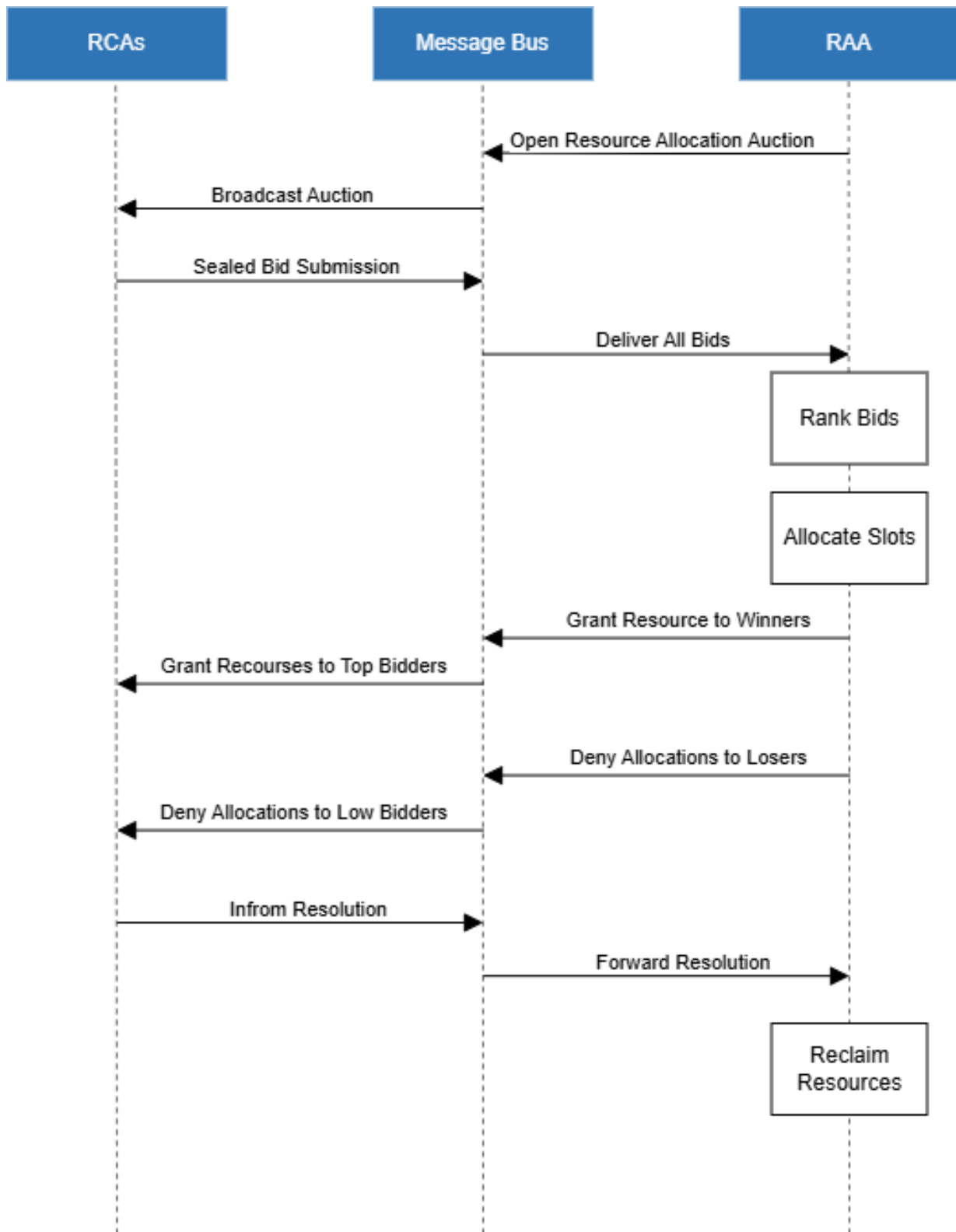


Figure 4 – Auction-Based Resource Allocation Sequence Diagram.

3.3.4 Voting Protocol for Quarantine

The sequence shows the coalition voting before executing a high-risk quarantine action.

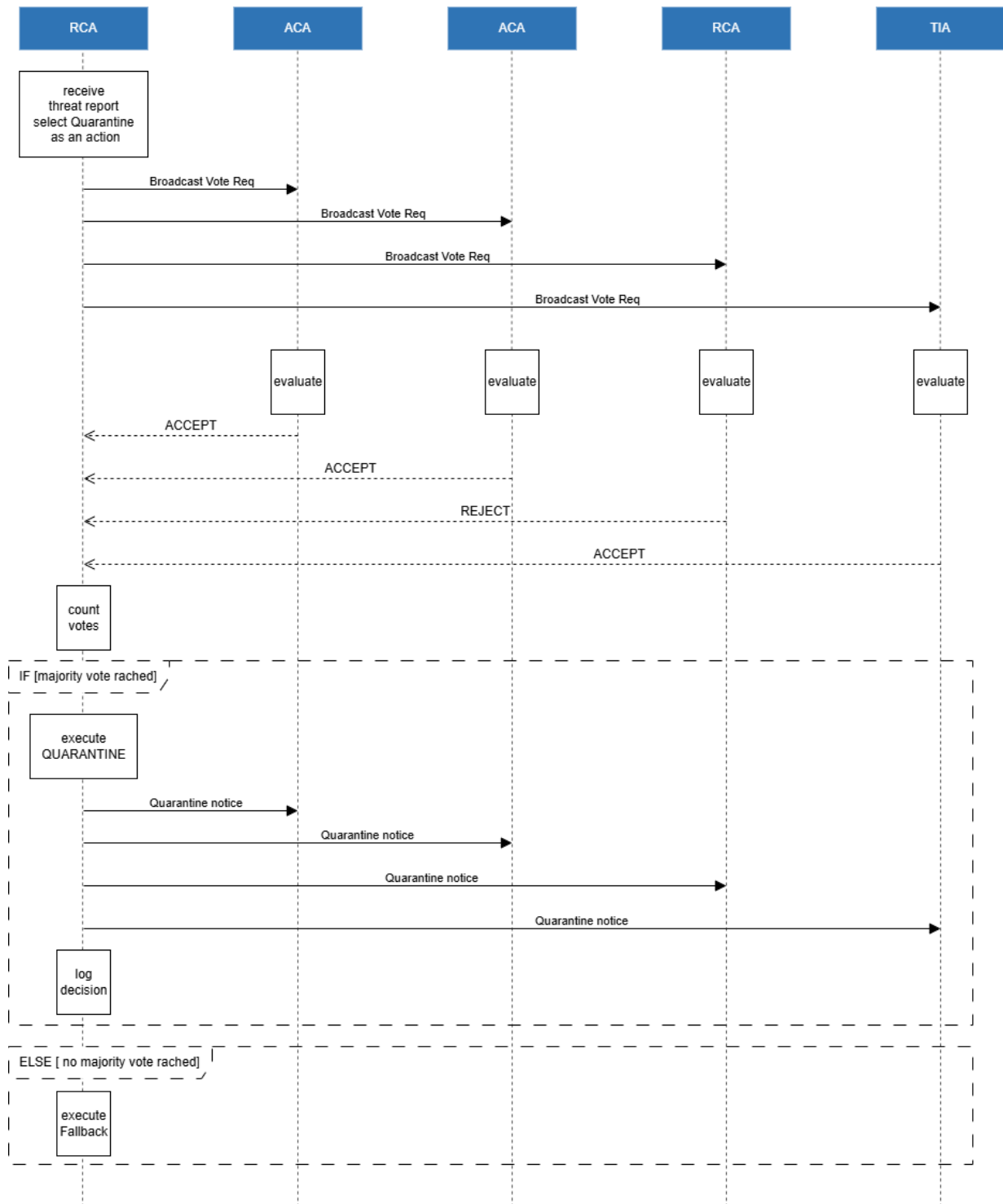


Figure 5 – Voting Protocol for Quarantine Sequence Diagram.

4 Coordination Mechanism Logic

4.1 Proportional Response Selection

The RCA must select the least disruptive action that is expected to neutralize a threat. This is implemented as a ranked action selection algorithm based on the threat severity score S and the estimated disruption score D of each available action.

Algorithm: `select_proportional_action( $S, A$ )`

Input: $S \in [0.0, 1.0]$ (Threat Severity), A (List of ResponseActions)
Output: ResponseAction to execute

```

1  $candidate\_actions \leftarrow \{a \in A \mid a.effective\_threshold \leq S\}$ 
2 if  $candidate\_actions$  is empty then
3   | return MONITOR
4 end
5 Sort  $candidate\_actions$  by  $disruption\_score$  ascending
6 return  $candidate\_actions[0]$ 
```

Figure 6 – Proportional Response Selection: $O(|A|)$ to filter and $O(|A| \log |A|)$ to sort candidate actions, bounded by the fixed action set size ($|A| \leq 5$), effectively $O(1)$.

4.2 Auction-Based Resource Allocation

The Resource Allocator uses a sealed-bid first-price auction to allocate limited defensive resources. All bids are submitted simultaneously, preventing agents from observing each other's bids before committing.

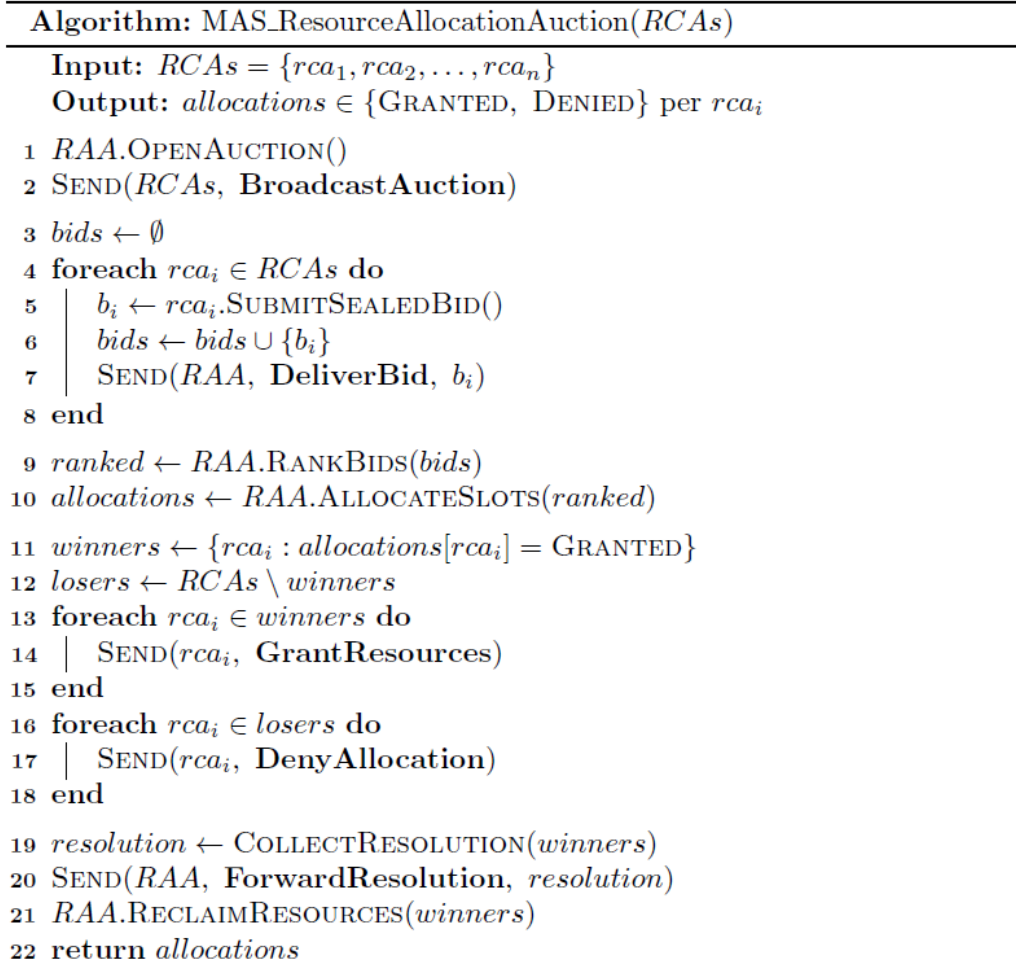


Figure 7 – Auction-Based Resource Allocation: $O(N \log N)$ to rank bids and $O(K)$ to allocate K slots, where $N \leq$ active RCAs (typically ≤ 4), effectively $O(1)$ in practice.

The sealed-bid first-price auction converges to a stable, deterministic allocation in a single round. This is guaranteed by three properties:

- **Single-round termination:** All bids are submitted simultaneously within a 300 ms window. The RAA collects bids, sorts them, and allocates in one pass (lines 9–10 of the algorithm). There is no iterative bidding, so the protocol terminates in exactly one round.
- **Deterministic outcome:** Bid values equal threat severity scores $S \in [0.0, 1.0]$. Ties are broken deterministically by (**severity**, **submitted_at**): the earlier submission wins. Because severity is a continuous float and simultaneous exact ties are negligible, the ranking is effectively strict. This produces a unique, reproducible allocation for any given input set.
- **Incentive alignment:** Agents have no incentive to misreport their bid value, because **bid_value** = **severity_score** is enforced by the message schema. This eliminates bid manipulation and makes the auction strategy-proof by design.

Given N bidders and K available resource slots ($K < N$), the allocation always assigns resources to the K highest-severity threats. The resulting allocation maximizes aggregate threat coverage, which is the system's primary social welfare objective. Allocation completes and results are broadcast within the 100 ms notification window because the sort is $O(N \log N)$ over a small N ($\leq$ number of active RCAs, typically ≤ 4).

4.3 Coalition Formation Protocol

Coalition formation is triggered automatically by the TIA when correlated multi-segment threats are detected. The goal is to form a minimal team that covers all segments of a multi-segment incident. The algorithm runs as follows:

Algorithm 1: MAS_CoalitionResponse(*network*)

Input: *network* = {*TMA*, *ACA*, *RCA*, *RAA*, *TIA*}
Output: *decision* ∈ {RESOLVED, UNRESOLVED}

```

1  alerts ← ∅
2  foreach TMAi do
3    ai ← TMAi.DETECTANOMALY()
4    alerts ← alerts ∪ {ai}
5  end

6  scores ← ∅
7  foreach ACAi do
8    si ← ACAi.CLASSIFYANDSCORE(alerts)
9    scores ← scores ∪ {si}
10   SEND(TIA, BusBroadcast, si)
11 end

12 context ← TIA.CORRELATESEGMENTS(scores)
13 candidates ← {ACA, RCA, RCA, RAA}
14 coalition ← ∅
15 foreach ai ∈ candidates do
16   SEND(ai, CoalitionInvitation, context)
17   wi ← ai.EVALUATEWORKLOAD()
18   if wi ≤ θload then
19     SEND(TIA, Accept)
20     coalition ← coalition ∪ {ai}
21   else
22     SEND(TIA, Reject)
23   end
24 end

25 leader ← RCA
26 TIA.CONFIRMCOALITION(coalition, leader)
27 resources ← SEND(RAA, RequestResources)
28 outcomes ← ∅
29 foreach ai ∈ coalition \ {leader} do
30   taski ← leader.ASSIGNRESPONSEACTION(ai, resources)
31   ri ← ai.EXECUTEACTION(taski)
32   outcomes ← outcomes ∪ {ri}
33   SEND(TIA, ReportOutcome, ri)
34 end

35 TIA.CONFIRMRESOLUTION(outcomes)
36 return RESOLVED

```

Figure 8 – Coalition Formation Protocol: $O(|candidates| \times |segments|)$ for the greedy cover loop, bounded by the fixed 5-agent topology and 4-segment network, effectively $O(1)$.

4.4 Voting-Based Quarantine Escalation Protocol

Quarantine actions carry high operational risk (service disruption). The RCA may not execute a quarantine unilaterally. Instead, it must obtain majority coalition approval via the following protocol. The vote window is strictly bounded at 300 ms to satisfy MTTR constraints.

4.4.1 Voting Protocol

Algorithm: VotingProtocol(*threat*, *coalition*)

Input: *threat* — ThreatReport with *severity* ≥ 0.7 and
action = QUARANTINE
coalition — set of active coalition member AgentIDs
Output: *decision* $\in$
{EXECUTE_QUARANTINE, FALLBACK_To_BLOCK}

```

1 vote_id  $\leftarrow$  GENERATEUUID()
2 deadline  $\leftarrow$  NOW() + 300ms
3 votes  $\leftarrow$   $\emptyset$ 
4 foreach agenti  $\in$  coalition do
5   | SEND(agenti, VoteRequest{
6   |   vote_id, QUARANTINE, threat.segment,
7   |   threat.id, threat.severity, deadline})
8 end
9 while NOW() < deadline do
10  | if VoteResponse received from agentj then
11  |   | votes[agentj]  $\leftarrow$  response.decision
12  |   end
13  | if |votes| = |coalition| then
14  |   | break
15  |   end
16 end
17 foreach agenti  $\in$  coalition where agenti  $\notin$  votes do
18  | votes[agenti]  $\leftarrow$  REJECT
19 end
20 accept_count  $\leftarrow$  |{v  $\in$  votes : v = ACCEPT}|
21 majority  $\leftarrow$   $\lfloor |coalition|/2 \rfloor + 1$ 
22 if accept_count  $\geq$  majority then
23  | LOG(vote_id, EXECUTE_QUARANTINE, votes)
24  | BROADCAST(coalition, QuarantineNotice{vote_id, threat.id, threat.segment})
25  | return EXECUTE_QUARANTINE
26 else
27  | LOG(vote_id, FALLBACK_To_BLOCK, votes)
28  | return FALLBACK_To_BLOCK
29 end

```

Figure 9 – Voting Protocol: $O(|coalition|)$ to broadcast and collect votes, where $|coalition| \leq N$ agents, with a hard 300 ms wall-clock deadline enforcing termination.

4.4.2 Voting Decision Rule Per Agent

Each coalition member evaluates the VoteRequest against its own local belief base:

Algorithm: EvaluateVoteRequest(*req*)

Input: *req* — VoteRequest
Output: *decision* $\in$ {ACCEPT, REJECT}

```

1 local_score  $\leftarrow$  belief_base.QUERY(threat_severity, req.segment)
2 if local_score  $\geq$  0.7 and local_workload  $<$   $\theta_{load}$  then
3   | return ACCEPT
4 else
5   | return REJECT
6 end

```

Figure 10 – Voting Decision Rule Per Agent: $O(1)$ — single belief base query against two conditions (*local_score* threshold and *local_workload* threshold).

4.5 Agent Failure & Resilience Protocol

Algorithm: handle_agent_failure(*F*)

Input: *F* (Failed Agent ID)

```

1 if no PING received from F for 1000ms then
2   | TIA.BROADCAST(CoverageRequest, F.segment)
3   | peer  $\leftarrow a_i \arg \min \{a_i.load \mid \text{ACCEPTSWITHIN}(a_i, 500ms)\}$ 
4   | REASSIGNCOVERAGE(F.segment, peer)
5   | assert elapsed_time  $\leq$  2000ms
6 end

```

Figure 11 – Agent Failure & Resilience Protocol: $O(N)$ to find the minimum-load peer willing to accept coverage within the 500 ms window, where $N \leq$ active agents per segment.

5 System Implementation Design

5.1 Technical Stack

Layer	Technology	Justification
Agent Runtime	Python 3.11 with asyncio	Native async/await for concurrent agent loops, rich ML ecosystem (scikit-learn, numpy).
Message Bus	Apache Kafka (or Redis Pub/Sub for lighter deployments)	High-throughput async pub/sub, topic-based routing; message persistence for replay mode.
ML / Classification	scikit-learn	Incremental / online learning support.
Simulation API	Python FastAPI service	RESTful interface for agents to read traffic data and execute actions, supports temporal scaling.
Visualization Backend	FastAPI + Web-Socket server	Real-time push of events to the dashboard frontend.
Visualization Frontend	React + D3.js + Recharts	Interactive network topology map, real-time charting; WebSocket consumer.
Logging	StructLog + PostgreSQL / SQLite / MySQL	Structured logs, queryable for post-run analysis and constraint checker.
Orchestration	Docker Compose	Containerized agents + bus + simulation, easy multi-instance scaling.

Table 3 – System Implementation Design — Technical Stack.

5.2 UML Class Diagram

The following describes the key classes and their relationships. All agents inherit from the abstract **BaseAgent** class, which provides the BDI lifecycle (perceive → reason → act), heartbeat emission, message bus integration, and logging.

5.2.1 BDI Hierarchy

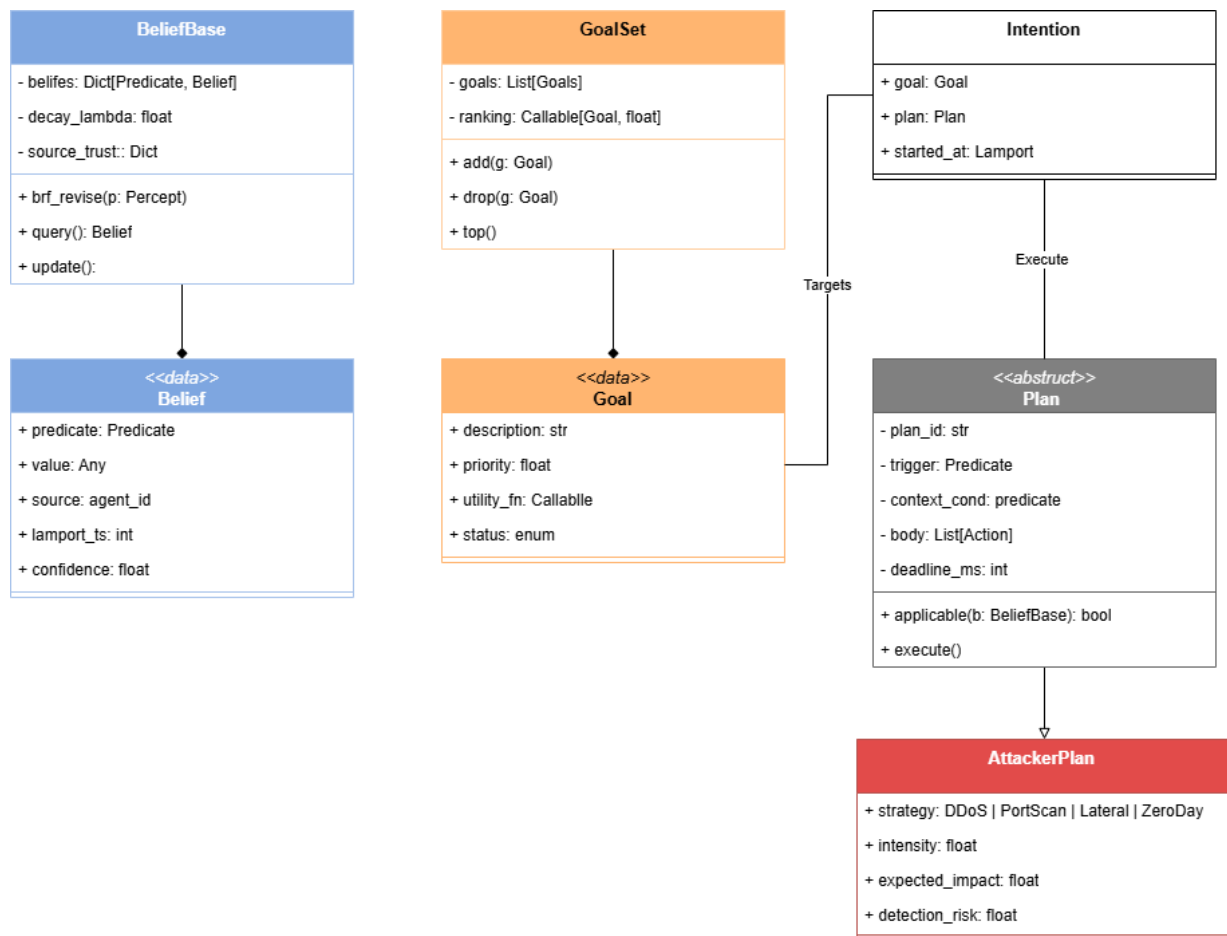


Figure 12 – BDI Agent Hierarchy Class Diagram.

5.2.2 Agent Class Diagram

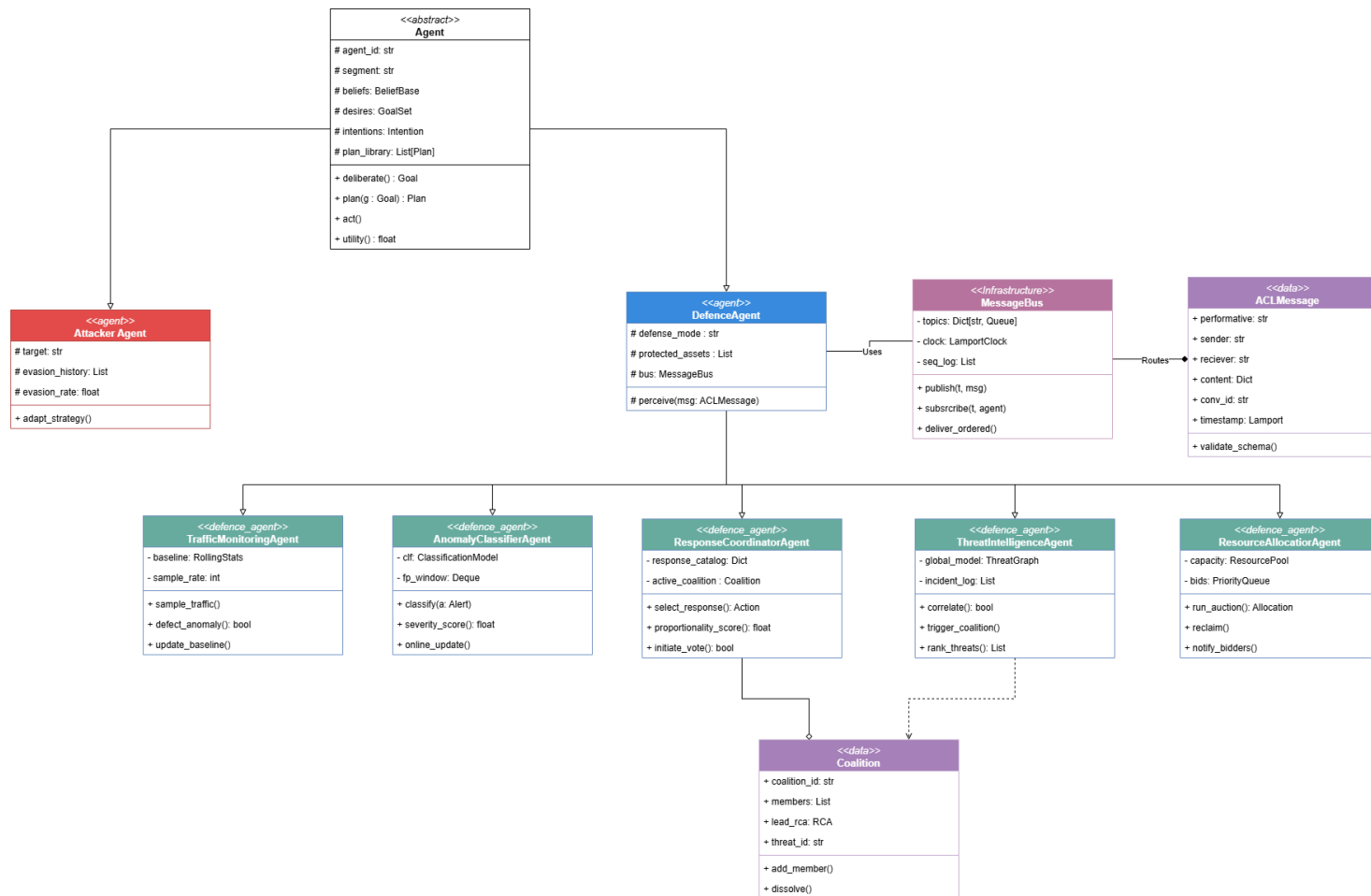


Figure 13 – FIPA-ACL Message Schema Class Diagram. Full diagram: [link](#).

5.3 Deployment Architecture

The system is deployed as a set of containerized services orchestrated via Docker Compose. Each agent type runs as an isolated container; all inter-agent communication is routed exclusively through the Message Bus container. No agent-to-agent direct connections exist.

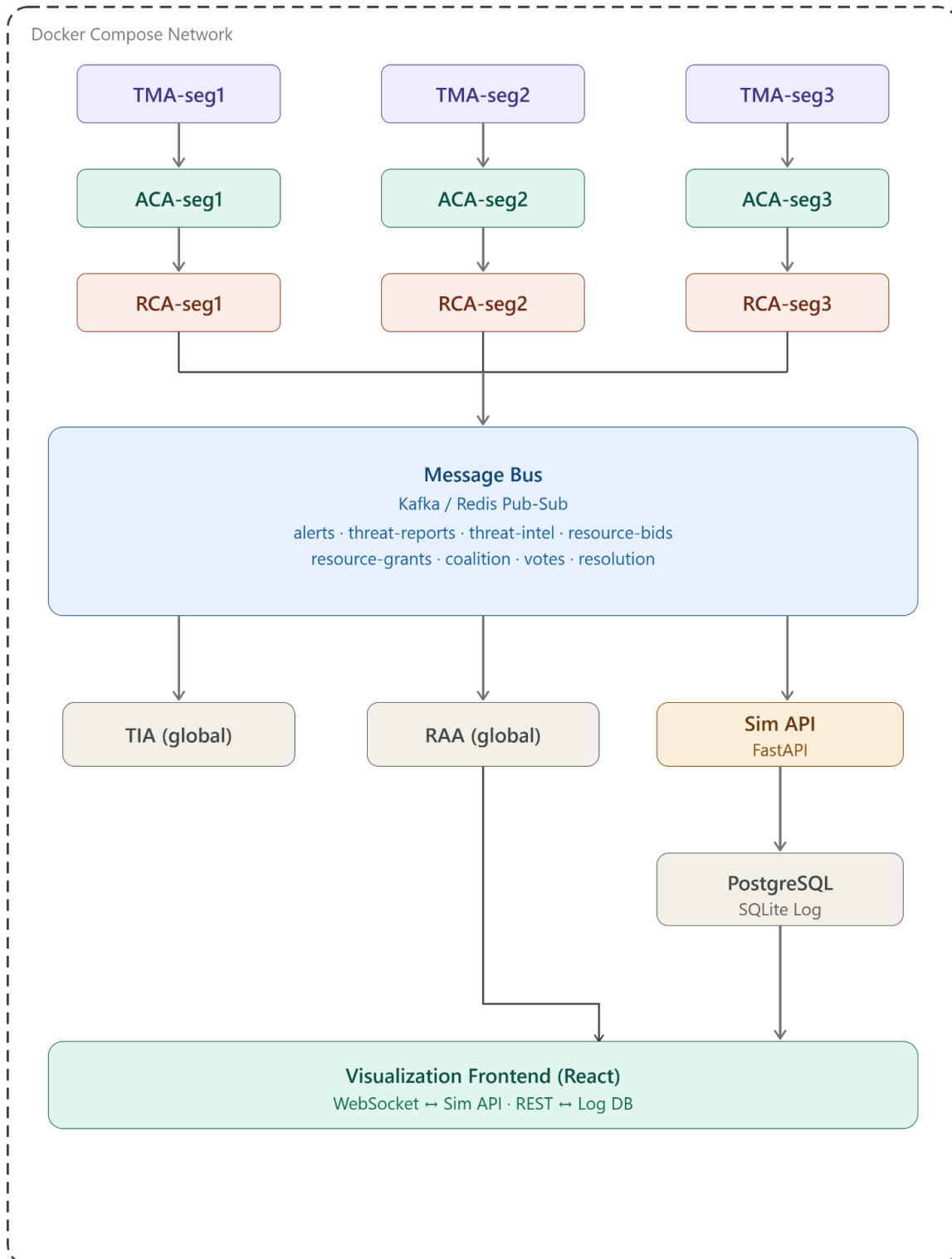


Figure 13 – Deployment Docker Architecture.

6 Simulator & Visualization Architecture

6.1 Simulation Engine Design

The simulation engine is a standalone Python service that models the four-segment network, generates synthetic traffic, injects attack scenarios, and exposes a REST API consumed by agents and the visualization dashboard.

6.1.1 Core Components

Component	Responsibility	Implementation
NetworkTopolog	Maintains graph of 4 segments and their connectivity rules.	Python dataclass; adjacency matrix for lateral movement path constraints.
TrafficGenerat	Produces realistic packet-level data per segment at configurable BPS/PPS rates.	Gaussian noise around configurable baseline, supports temporal scaling via clock multiplier.
AttackInjector	Injects attacker agent actions into the traffic stream at scheduled or manual trigger.	Drives injection; supports DDoS, port scan, lateral movement, zero-day patterns.
SimClock	Controls simulation time (real-time or 0.5× to 10× accelerated).	asyncio event loop + configurable tick rate; all timestamps normalized to sim time.
StateManager	Tracks current segment health, active defenses, resource utilization.	In-memory dict updated by agent action calls; queried by visualization WebSocket feed.
REST API	/packets, /topology, /state, /action, /metrics, /inject-attack.	FastAPI, authenticated with agent tokens; rate-limited per agent to enforce resource overhead cap.

Table 4 – *Simulation Engine Design — Core Components.*

6.1.2 Simulation API Endpoints

Endpoint	Method	Caller	Description
/packets/{segment}	GET	TMA	Returns last N packets for segment.
/action	POST	RCA	Applies response action {type, segment, params}; returns effectiveness estimate.
/topology	GET	TIA, Visualizer	Returns current 4-segment topology with node health states.
/state	GET	Visualizer, RAA	Returns full system state snapshot: segment health, agent status, resource utilization.
/metrics	GET	Visualizer, analytics	Returns current DR, FPR, MTTR, availability, SW values.
/inject-attack	POST	Test harness	Manually injects an attack scenario by type and target segment.

Table 5 – *Simulation API Endpoints.*

6.2 Visualization

6.2.1 Dashboard Page

The dashboard is a React page application that connects to the simulation engine via WebSocket for real-time state streaming and to the agent logging service via REST for historical queries.

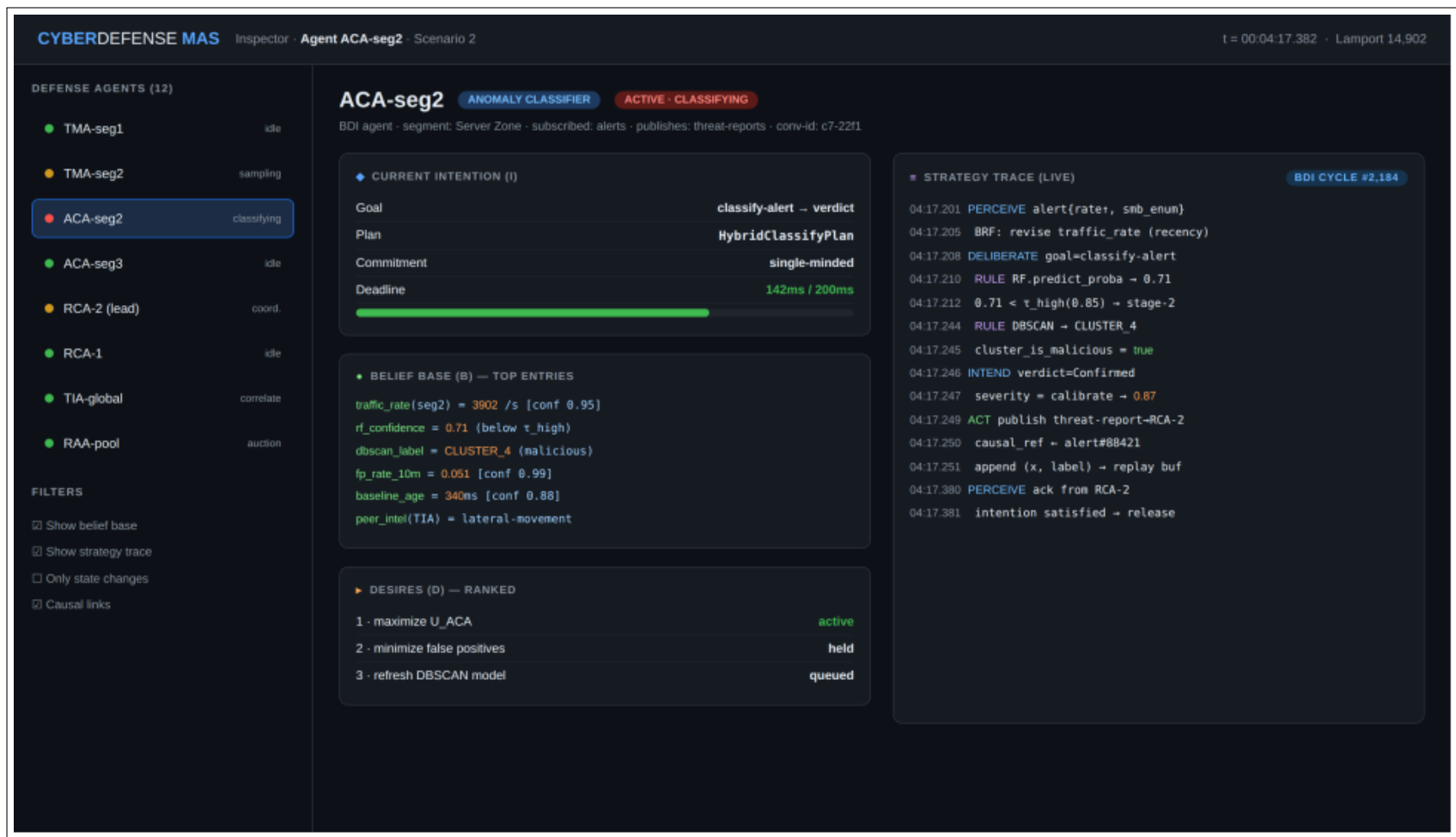
*Figure 15 – Dashboard Page.*

The real-time visualization dashboard serves as the central interface for observing, verifying, and auditing the system's state. The screen is divided into five functional zones that map agent behavior directly against system requirements:

- **Network Topology (Left):** Dynamically tracks the four logical network segments, visually flagging the *Server Zone* as “Under Attack”.
- **Agent Message Flow & Coalition Canvas (Center):** Uses a dashed overlay to highlight the dynamically instantiated Coalition. Vector lines trace raw FIPA-ACL asynchronous message performatives across the pub/sub bus.
- **Live Alert & Threat Feed (Right):** System alerts shown chronologically, color-coded by priority (critical, suspicious, or informational).
- **Live Performance Metrics (Bottom Left):** Graphically validates global social welfare against mandatory criteria, displaying a compliant Detection Rate (target > 90%), a False Positive Rate (target < 8%), and a MTTR (target < 1,000 ms).
- **Resource Allocation (Bottom Right):** Monitors host stability, confirming that MAS host overhead is safely bounded well below the critical 40% capacity cap.

6.2.2 Agent Inspector Page

The agent inspector lets an examiner select any agent and see its full BDI internal state in real time: current intention and committed plan, the live belief base with confidence values, ranked desires, and the human-readable strategy trace required for verification.

*Figure 16 – Agent Inspector Page.*

The panel breaks down as follows:

- **Current Intention (I):** Displays the agent's active execution plan and current goal commitment. It features a dynamic countdown bar that tracks execution latency, allowing observers to verify runtime compliance with mandatory operational constraints and deadlines.
- **Belief Base (B):** Houses the agent's localized environmental knowledge. This includes real-time data, local model confidence scores, and synchronized threat intelligence shared by peer agents across the decentralized network.
- **Desires (D):** Outlines the agent's prioritized motivational stack. It distinguishes between active, held, and queued goals to illustrate how the agent dynamically balances its operational objectives and utility optimization.
- **Strategy Trace (Live):** Emits a chronological, auditable reasoning log for the current BDI cycle. It charts the step-by-step cognitive loop from initial environmental perception and belief revision through logical deliberation and algorithm evaluation to the final execution of defensive actions.

6.2.3 Validator Page

The real-time Replay & Constraint Validation dashboard provides observers with an offline verification and auditing environment to evaluate system integrity against core specifications.

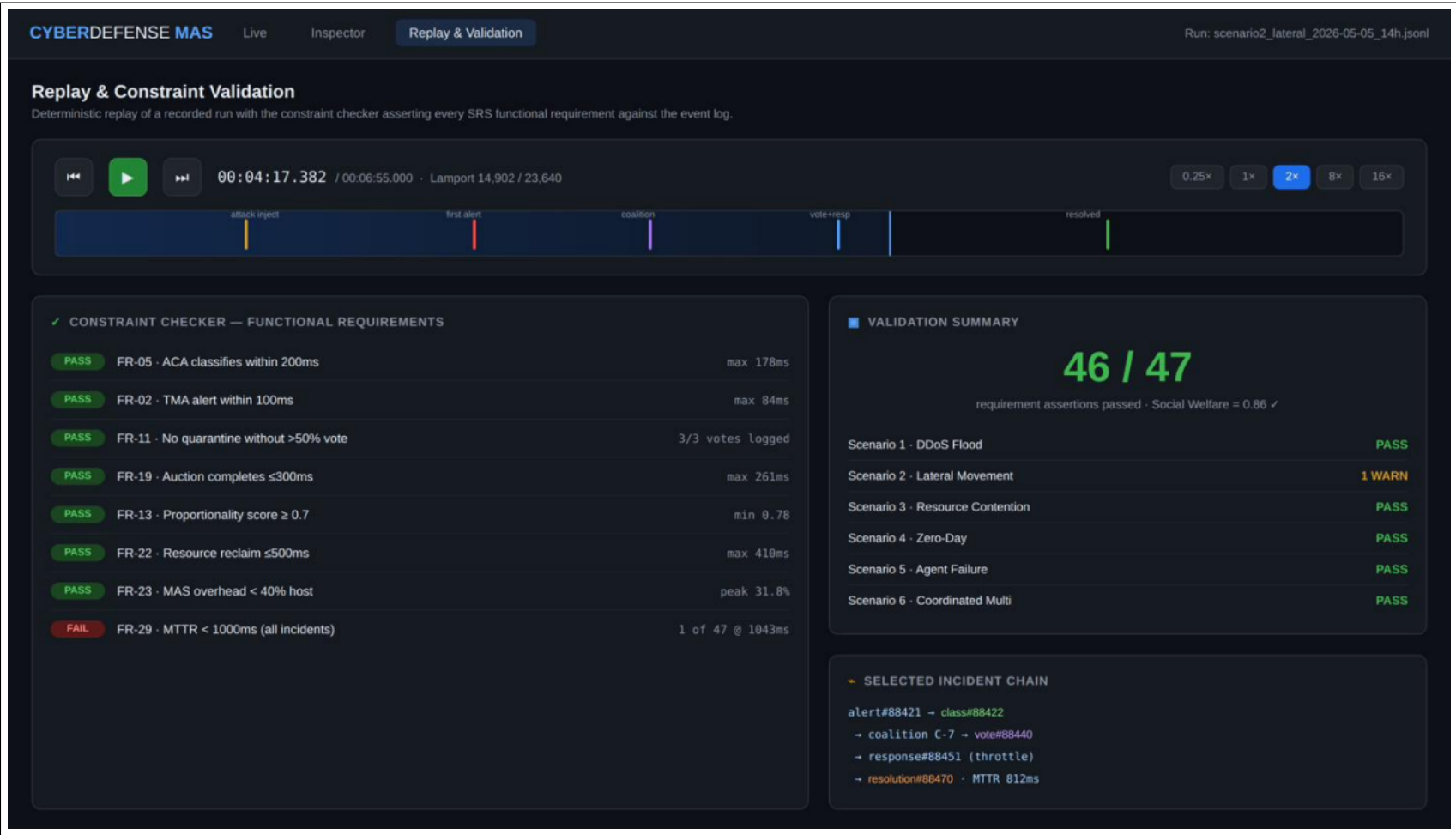


Figure 17 – Validator Page.

It cross-examines historical event logs against system rules through four primary functional zones:

- **Playback Controls & Event Timeline (Top):** Features interactive speed scaling and an interactive event scrubber. This allows observers to trace the exact chronological progression of a simulation run across major system milestones like threat injections, alert triggers, coalitions, and resolutions.
- **Constraint Checker (Left Panel):** Functioning as an automated compliance validator, this panel parses the underlying system logs to evaluate agent actions directly against design requirements. It renders immediate pass/fail flags alongside observed operational boundaries to ensure performance constraints like execution deadlines, resource caps, and voting protocols are strictly honored.
- **Validation Summary (Top Right Panel):** Aggregates the macro-performance of the multi-agent framework across the entire execution run. It quantifies overall requirement assertions, evaluates the final global Social Welfare score, and maps validation status sequentially across system stress-test scenarios.
- **Selected Incident Chain (Bottom Right Panel):** Explicitly maps the transactional causality of a specific threat lifecycle. It visually links the continuous defensive pipeline from its initial ingestion alert, through agent classification, coalition delegation, resource voting, response execution, and eventual state synchronization.

7 Data & Analytics Design

The system uses structured JSON logging throughout. Every agent action, inter-agent message, and environment state change is recorded to a persistent event store, enabling both real-time visualization and post-run analysis.

7.1 Event Log Schema

```
{
  "event_id":      "uuid",
  "lamport_ts":    int,
  "wall_ms":       float,
  "event_type":    "ALERT_PUBLISHED | THREAT_CLASSIFIED |
                  ACTION_EXECUTED | AUCTION_COMPLETED | VOTE_CAST |
                  COALITION_FORMED | AGENT_FAILED |
                  RESOURCE_ALLOCATED | INCIDENT_RESOLVED",
  "timestamp":     "2026-05-05T10:00:00.000Z",
  "agent_id":      str,
  "agent_type":    str,
  "segment":       str,
  "payload":       { <event-type-specific fields> },
  "latency_ms":    int,
  "decision_trace":{
    "inputs":       {...},
    "plan_selected": "classify_alert",
    "reasoning":     "severity_score=0.91 > threshold",
    "action":        "PUBLISH_THREAT_REPORT"
  }
}
```

7.2 Metric Collection Design

Metric	Computation
Detection Rate (DR)	$\frac{TP}{TP + FN}$
False Positive Rate (FPR)	$\frac{FP}{FP + TN}$
$MTTR_{Alert}$	$T_{Publish} - T_{detect}$
$MTTR_{Response}$	$T_{executed} - T_{report}$
System Availability	$\frac{\text{Total Time} - \text{Disrupted Time}}{\text{Total Time}}$
Resource Overhead	$\frac{\text{MAS CPU}}{\text{Total Host CPU}}$
Social Welfare (SW)	$SW = w_{TMA} \cdot U_{TMA} + w_{ACA} \cdot U_{ACA} + w_{RCA} \cdot U_{RCA} \\ + w_{RAA} \cdot U_{RAA} + w_{TIA} \cdot U_{TIA}$ The weights of each agent are defined in the SRS file.
Attacker Utility (U_{ATK})	$\text{Disruption} \times \text{Evasion} \times \left(1 - \frac{1}{MTTR_Response}\right)$

Table 6 – Metric Collection Design.

7.3 Validation Scenario Reporting

The analytics module generates a Scenario Report containing the following sections:

- **Scenario Metadata:** `scenario_id`, `start_time`, `duration`, attack types injected.
- **Agent Performance Table:** per-agent utility values (U_{TMA} , U_{ACA} , U_{RCA} , U_{RAA} , U_{TIA}).
- **System Metrics Summary:** DR, FPR, $MTTR_{Alert}$, $MTTR_{Response}$, Availability, Resource Overhead.
- **Social Welfare Score:** SW value with PASS/FAIL against ≥ 0.80 threshold.
- **Attacker Utility:** U_{ATK} value with PASS/FAIL against target.
- **Constraint Violations:** list of any FR violations detected by ConstraintChecker.

- **Strategy Traces:** full decision log for each agent, exportable as JSON.

7.4 Scenario Success Criteria Summary

Scenario	Key Defense Criteria	Attacker Crite- rion	Social Welfare
Single DDoS	$DR > 90\%$, $MTTR_{\text{Response}} < 1000\text{ ms}$, Availability $> 99\%$	$U_{\text{ATK}} < 0.2$	$SW \geq 0.8$
Multi-Segment	Coalition formed $< 1\text{ s}$, simultaneous responses	EvasionRate < 0.15	$SW \geq 0.8$
Resource Con- tention	Auction complete $< 300\text{ ms}$, CPU + Mem $\leq 40\%$	N/A	$SW \geq 0.8$
Zero-Day	Novel attack detected $< 500\text{ ms}$, FPR $< 10\%$	N/A	$SW \geq 0.8$
Agent Failure	Coverage reassigned $< 2\text{ s}$, $MTTR_{\text{Response}} < 1000\text{ ms}$	N/A	$SW \geq 0.8$
Voting Protocol	Vote cycle $< 300\text{ ms}$, action follows majority strictly	N/A	$SW \geq 0.8$

Table 7 – Scenario Success Criteria Summary.

8 Summary

In conclusion this document presents the complete technical design for the Collaborative Cyber-Defense and Automated Incident Response Multi-Agent System. The system leverages a BDI cognitive architecture across five specialized agent types coordinated through FIPA-ACL messaging, sealed-bid auctions, coalition formation, and voting-based consensus to deliver a principled, adaptive defense against a diverse threat landscape. Each coordination mechanism is formally specified and tied directly to measurable performance targets, ensuring the system remains verifiable and auditable throughout its operation. The design prioritizes both responsiveness and resilience: strict latency budgets govern every pipeline stage from anomaly detection to defensive action, while Lamport clocks, idempotent deduplication, and an agent failure protocol guard against race conditions and partial failures. Taken together, the architecture provides a robust foundation for simulation, validation, and future extension, with the visualization dashboard and constraint checker enabling observers to verify in real time that system behavior meets the Social Welfare threshold and all functional requirements defined in this document.

9 Appendix: Glossary & Acronyms

Term / Acronym	Definition
BDI	Belief-Desire-Intention: A cognitive architecture for rational agents where Beliefs represent the agent's knowledge state, Desires represent its goals, and Intentions are the plans it commits to executing.
FIPA-ACL	Foundation for Intelligent Physical Agents — Agent Communication Language: A standardized protocol for inter-agent messaging defining message performatives, structure, and semantics.
MAS	Multi-Agent System: A system composed of multiple autonomous agents interacting within a shared environment.
MTTR	Mean Time to Respond: The average elapsed time from attack detection to successful mitigation. Split into MTTR_Alert (TMA to alert publication) and MTTR_Response (detection to defensive action).
TMA	Traffic Monitor Agent: Monitors raw network traffic in an assigned segment and publishes anomaly alerts.
ACA	Anomaly Classifier Agent: Classifies alerts from TMAs as Normal, Suspicious, or Confirmed Threat using ML models.
RCA	Response Coordinator Agent: The incident commander; selects and executes proportional defensive responses.
TIA	Threat Intelligence Agent: Maintains the global threat model, detects multi-segment correlations, and coordinates coalition formation.
RAA	Resource Allocator Agent: Manages shared defensive resources via a sealed-bid auction mechanism.
DDoS	Distributed Denial-of-Service: An attack that floods a target with traffic from many sources to exhaust bandwidth or computational resources.
DPI	Deep Packet Inspection: A network analysis technique that examines packet payloads (not just headers) to detect malicious content.
SW	Social Welfare: The weighted aggregate utility score of all defense agents, used as the primary system-level success metric. Target: $SW \geq 0.80$.
EMA	Exponential Moving Average: A statistical method for updating a rolling baseline that weights recent observations more heavily than older ones.
Telemetry	A stream of structured measurement events (flows, alerts, actions) continuously emitted by agents and the network environment.
Strategy Trace	A per-decision reasoning log tracking inputs received, rules or utility applied, and the chosen action to allow observer verification.
BRF	Belief Revision Function: The internal agent component that updates the belief base upon receiving a new percept using recency and decay rules.
Lamport Clock	A logical counter providing a partial-order timestamp across distributed agents, ensuring total message ordering on the bus with deterministic tie-breaking.
Pub/Sub	Publish-Subscribe: A decoupled messaging pattern where agents publish data to named topics and subscribe to them without needing to know peer identities.
Coalition	A time-bounded, goal-oriented team of defense agents formed dynamically to address a coordinated multi-segment attack.
Contract-Net	An interaction protocol where a manager agent broadcasts an invitation and evaluates peer responses, used here to coordinate coalition formation.
Sealed-Bid First-Price Auction	An auction format where bidders submit private bids simultaneously, and the highest bidder wins the resource at the price of their own bid.

Term / Acronym	Definition
DR / FPR	Detection Rate and False Positive Rate: The two primary performance metrics used to evaluate classifier accuracy and social welfare safety thresholds.
DPI Slot	Deep Packet Inspection Capacity Unit: The granular, scarce computational resource managed and distributed by the Resource Allocator Agent.
causal_ref	Event back-link reference that connects an effect (such as a mitigation response) back to its root cause (the initial anomaly alert), enabling end-to-end incident chain reconstruction.
Multi-Segment Incident	A coordinated cyber-threat that simultaneously targets or manifests across two or more distinct network zones (such as the Public-Facing Services and the Internal User Subnet). These incidents bypass local security boundaries, triggering correlation analysis by the Threat Intelligence Agent (TIA) and necessitating a multi-agent coalition to execute a synchronized defense strategy.

Table 8: Glossary and Acronyms.